

Projet en groupe de 3 : Système de gestion de bibliothèque en microservices

Objectif général

Construire une architecture microservices complète comprenant :

- Trois microservices métier : **service-livre**, **service-emprunteur**, **service-emprunt** ;
- Un serveur Eureka pour la découverte de services ;
- Une API Gateway pour le routage unifié.

Chaque microservice possède sa propre base de données MySQL, expose une API REST et utilise des DTO ainsi que des entités JPA. Tous les services doivent s'enregistrer dans Eureka et communiquer via leurs noms logiques.

Contraintes techniques

- Spring Boot 4.x / Spring Cloud 2024.x
- Java 17+
- Maven
- MySQL (une base différente par service)
- Communication inter-services possible par appels REST (client HTTP)

1 Partie 1 – Serveur Eureka

- 1.1** Créez un projet Spring Boot nommé **eureka-server** avec la dépendance **Eureka Server**. Quelle annotation devez-vous ajouter à la classe principale pour activer le serveur ?
- 1.2** Configurez le fichier **application.properties** pour que le serveur tourne sur le port 8761 et ne tente pas de s'enregistrer lui-même. Quelles propriétés faut-il définir ?
- 1.3** Démarrez le serveur. Quelle URL permet de visualiser la console Eureka ? Que devez-vous y voir avant le lancement de tout microservice ?
- 1.4** Expliquez brièvement le rôle du mécanisme de *heartbeat* dans Eureka.

2 Partie 2 – Microservice service-livre

- 2.1 Créez un projet Spring Boot nommé `service-livre` avec les dépendances : Spring Web, Spring Data JPA, MySQL Driver, Eureka Discovery Client, Lombok, Spring Boot DevTools. Quel nom logique (`spring.application.name`) donnez-vous au service ? Sur quel port le faites-vous tourner ?
- 2.2 Créez une base MySQL nommée `bibliotheque_livre`. Écrivez la configuration de connexion dans `application.properties`. Expliquez l'utilité de `spring.jpa.hibernate.ddl-auto`.
- 2.3 Modélisez une entité JPA `Livre` avec les attributs : `id` (auto-généré), `titre`, `auteur`, `isbn`, `disponible` (boolean). Ajoutez les annotations Lombok.
- 2.4 Créez un DTO `LivreDTO` (champs identiques à l'entité mais sans annotations JPA). Pourquoi est-il recommandé d'utiliser des DTO plutôt que les entités dans les contrôleurs ?
- 2.5 Implémentez un `LivreRepository` (héritant de `JpaRepository`). Écrivez un `LivreService` contenant les méthodes : `findAll()`, `findById(id)`, `save(dto)`, `delete(id)`. Pensez à la conversion entité DTO.
- 2.6 Réalisez un `LivreController` exposant les endpoints :
 - GET `/api/livres`
 - GET `/api/livres/{id}`
 - POST `/api/livres`
 - DELETE `/api/livres/{id}`
- 2.7 Ajoutez l'annotation `@EnableDiscoveryClient` sur la classe principale. Lancez le service après avoir lancé Eureka. Vérifiez dans la console Eureka que le service apparaît. Que constatez-vous ?
- 2.8 Proposez un endpoint supplémentaire PUT `/api/livres/{id}/disponibilite` permettant de modifier la disponibilité d'un livre (utile pour `service-emprunt`).

3 Partie 3 – Microservice service-emprunteur

- 3.1 Créez `service-emprunteur` sur le port 8082. Sa base de données s'appellera `bibliotheque_emprunt`. Définissez l'entité `Emprunteur` (`id`, `nom`, `email`, `téléphone`) et son DTO.
- 3.2 Implémentez le repository, le service et le contrôleur pour exposer :
 - GET `/api/emprunteurs`
 - GET `/api/emprunteurs/{id}`
 - POST `/api/emprunteurs`
 - DELETE `/api/emprunteurs/{id}`
- 3.3 Activez l'enregistrement dans Eureka. Testez le démarrage et vérifiez l'apparition dans la console.

4 Partie 4 – Microservice service-emprunt (appels inter-services)

- 4.1 Créez `service-emprunt` sur le port 8083 avec sa base `bibliotheque_emprunt`. L'entité `Emprunt` contiendra : `id`, `dateEmprunt` (`LocalDate`), `dateRetourPrevue`,

`dateRetourReelle`, `livreId` (Long), `emprunteurId` (Long), `statut` (String: EN_COURS, RENDU, RETARD). Créez le DTO correspondant.

- 4.2 Avant de créer un emprunt, il faut vérifier que le livre (via `livreId`) existe et est disponible, et que l'emprunteur (via `emprunteurId`) existe. Expliquez comment vous allez interroger `service-livre` et `service-emprunteur` en utilisant leur nom logique (fourni par Eureka) plutôt qu'une URL en dur.
- 4.3 Implémentez un client REST (par exemple avec `RestClient` ou `WebClient`) dans `service-emprunt`. Quelle configuration est nécessaire pour que le client utilise la résolution de noms via Eureka ?
- 4.4 Créez un endpoint POST `/api/emprunts` qui :
 - Reçoit un `EmpruntDTO` (sans id, avec `livreId` et `emprunteurId`).
 - Vérifie l'existence du livre et sa disponibilité.
 - Vérifie l'existence de l'emprunteur.
 - Crée l'emprunt avec `dateEmprunt = aujourd'hui`, `dateRetourPrevue = aujourd'hui + 15 jours`, `statut = EN_COURS`.
 - Met à jour le livre correspondant en passant `disponible = false` (appel au endpoint PUT du `service-livre`).
- 4.5 Ajoutez un endpoint PUT `/api/emprunts/{id}/retour` qui :
 - Change le statut de l'emprunt à RENDU.
 - Renseigne `dateRetourReelle` avec la date du jour.
 - Remet `disponible = true` sur le livre associé.
- 4.6 Testez manuellement les trois microservices (ordre : Eureka → livre → emprunteur → emprunt). La communication inter-services fonctionne-t-elle ? Décrivez les éventuels problèmes rencontrés.

5 Partie 5 – API Gateway

- 5.1 Créez un projet `gateway-service` avec les dépendances `Gateway` et `Eureka Discovery Client`. Configurez-le pour qu'il s'enregistre dans Eureka (port 8080).
- 5.2 Dans `application.yml`, activez la découverte automatique des routes en ajoutant :

```
spring.cloud.gateway.discovery.locator.enabled=true
spring.cloud.gateway.discovery.locator.lower-case-service-id=
true
```

Sans écrire de routes personnalisées, comment la Gateway redirige-t-elle les requêtes ? Donnez un exemple d'URL pour appeler `service-livre` via la Gateway.

- 5.3 Écrivez des routes explicites pour chacun des trois services (par exemple `/api/livres/**` → `lb://service-livre`). Quelle est l'utilité du préfixe `lb://` ?
- 5.4 Ajoutez un filtre global qui ajoute un en-tête `X-Gateway-Id` avec la valeur `my-gateway` à chaque requête sortante. Expliquez comment faire.
- 5.5 Redémarrez tous les services (Eureka → microservices → Gateway). Testez l'accès à `http://localhost:8080/api/livres`. Décrivez le cheminement complet de la requête.

6 Partie 6 – Questions théoriques

Répondez aux questions suivantes de manière argumentée :

- 6.1 Quelle est la différence fondamentale entre le rôle d'Eureka Server et celui de l'API Gateway ?
- 6.2 Que se passe-t-il si un microservice tombe en panne ? Comment Eureka réagit-il ? La Gateway continue-t-elle d'essayer d'y accéder ?
- 6.3 Proposez une amélioration pour rendre le système plus résilient (timeouts, retries, circuit breaker) au niveau de la Gateway ou des clients HTTP.
- 6.4 Comment pourrait-on sécuriser l'ensemble des services en ajoutant une authentification par jeton JWT au niveau de la Gateway ?
- 6.5 Expliquez en quelques lignes le principe de *Service Discovery* et pourquoi il est essentiel dans une architecture microservices.

Modalités de rendu

Chaque étudiant doit fournir :

- Un dépôt Git (ou une archive) contenant les 5 projets : `eureka-server`, `service-livre`, `service-emprunteur`, `service-emprunt`, `gateway-service`.
- Un fichier `README.md` expliquant :
 - L'ordre de lancement des services.
 - Les endpoints disponibles avec des exemples de requêtes (Postman).
- Les réponses aux questions théoriques (Partie 6) dans un document séparé ou intégrées au `README`.

NB : aucune correction ni solution n'est fournie dans cet énoncé. Chaque étudiant doit réaliser l'exercice par lui-même et non générer par une IA. Un étudiant par service